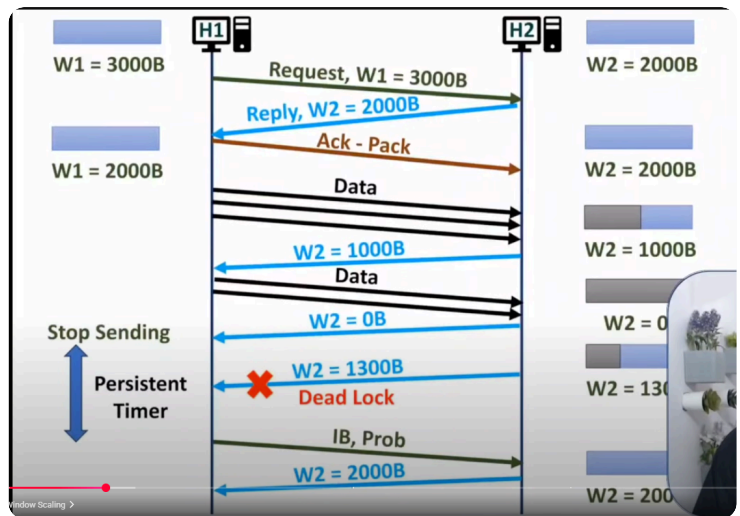


TCP Flow Control Explained



0:01

Hello friends.

0:02

Welcome to Engineering Funda family.

0:05

In this video I am going to explain you flow control mechanism in TCP.

0:10

At the very beginning of this chapter in transport layer, I have told you flow control mechanism that

0:17

is done in transport layer.

0:18

If you have seen my first video based on this chapter of transport layer, in that I have explained

0:24

you how we have flow control mechanism.

0:29

Right?

0:29

And here we have transmitter and receiver.

0:32

By this example I have explained you what is flow control.

0:35

flow control is a control of data transmission speed.

0:40

For example, if transmitter is sending data with the speed of 10 Mbps, but as if receiver can handle

0:47

up to 1 Mbps only, then this receiver cannot receive data which is forwarded by transmitter over

0:53

here.

0:54

So here receiver will give feedback to transmitter.

0:57

In that feedback receiver will say I can handle up to one Mbps only, so you will have to transmit data

1:04

with speed at max with one Mbps.

1:07

So this is how flow control is happening.

1:10

There can be another scenario, even like as if transmitter is sending data with ten Mbps speed.

1:16

But if receiver can handle up to 100 Mbps speed, then the receiver will give feedback to transmitter.

1:22

It will say like I can handle up to 100 Mbps.

1:25

You are slow right now.

1:27

So this is how flow control mechanism that is happening, in which we will be controlling data rate

1:34

in between two computers.

1:36

Right.

1:37

So here in TCP, how it happens, that is what the case which I am going to explain you in this video.

1:43

So here when it comes to flow control, first of all you need to understand how many basics are there.

Basics of Flow Control in TCP Protocol

1:49

See flow control explains how transfer speed of data is adjusted in between two computers during data

1:57

transmission.

1:58

So as if one computer is slower compared to another.

2:02

In that case, there has to have adjustment of speed, and the second scenario is even possible if data

2:08

transmission is happening with lower speed, and if resources are having higher capabilities, then

2:15

data transmission speed that can be increased over here.

2:18

See, both of the possibilities are possible.

2:21

And in TCP to maintain flow control see these three tools.

2:26

That is very very essential.

2:28

First is window scaling.

2:30

Second is advertisement window size and third is persistent timer.

2:34

I will explain you these three tools, along with practical examples that will gives you more clarity

2:40

about how TCP handles flow control.

2:44

Let me explain you that by one practical scenario.

2:47

So here see we are having two computers H1 and H2.

Window Scaling

2:52

And in TCP you should know what we do is we do connection establishment at the very beginning.

2:58

That is the first stage.

3:00

Second stage is data transfer and third stage is connection termination.

3:05

Right.

3:05

So in connection establishment both of these computers will be performing three way handshake.

3:13

So there will be three segments that is getting forwarded over here in connection establishment.

3:18

You see this is first segment that is requested by H one.

3:24

This is second segment.

3:25

That is a reply to the segment forwarded by H one right by H two.

3:32

And then there is acknowledgment.

3:35

So by three segments that is connection establishment.

3:39

And in that what is happening you see when H one is forwarding request to H two, at that time it will

3:46

mention window size available at H one.

3:51

see window size explains you how much data that can be handled at H1, right?

3:58

How much size of buffer is available at H1 that is mentioned by window size?

4:04

Then H2 will give reply in which it will give window size details of H2.

4:11

So here you see H1 is saying my window size is 3000.

4:16

And then h2 is saying okay my window size that is 2000.

4:20

So now h1 and h2 both knows what are the possible window size that is there in between these two computers.

4:28

So here what will happen is both of these computer will agreed upon lower of this two window size.

4:35

So H1 is having 3000 H2 is having 2000.

4:39

But after this they will get to know like okay we should be having 2000 window size as minimum of these

4:46

two should get selected over here.

4:48

Right.

4:49

So now you see at H1 window size that is 2000 and H2 window size is 2000 Right now you see what is happening

4:59

here.

4:59

Here there is scaling of window size that we have done.

5:04

Initially H1 was having capability up to 3000 byte of window size and H2 was having 2000.

5:11

But after connection establishment they have scaled their window size to 2000 bytes.

5:18

Right.

5:18

So that is what the first key aspect of TCP flow control like.

5:23

They will communicate with each other regarding their window size.

5:27

And based on that they can understand okay, by that size only we should be transferring data.

5:34

Now you see what is happening.

5:36

Secondly, now after connection establishment H1 is communicating with H2.

Advertisement Window Size

5:42

Here let us consider.

5:44

H1 is forwarding data to H2.

5:47

So you see H1 is forwarding some data to H2.

5:51

So as this data that is getting received by H2, its window size that will be getting filled up.

5:57

Right.

5:58

So what will happen is you see here 1000 bytes that is there inside this window.

6:05

So only 1000 bytes are available.

6:09

1000 bytes.

6:10

That is getting buffered over here in this window.

6:12

So only 1000 bytes are available.

6:15

So here you see this H2 is giving feedback to H1 regarding available window size.

6:21

That is 1000 bytes only.

6:24

So this is what window advertisement like initially they have agreed upon with 2000 byte of window size.

6:32

But because of data that is getting buffered in this window, only 1000 byte is available.

6:39

So this H2 is giving advertisement window size to H1.

6:45

It indicates right now only 1000 byte is available with this window.

6:49

The reason is 1000 byte is filled, right?

6:53

Then H1 is forwarding data, and once that complete window that is getting filled over here, this H

7:02

two will give again.

7:03

Advertisement.

7:04

Window size.

7:05

And it says to H one my window is full.

7:10

Zero byte is available right now.

7:12

The reason is entire window that is full over here because of I have received data from you.

7:19

Right.

7:19

So zero byte is available.

7:21

So here this H one will stop sending the data.

7:25

Why?

7:25

The reason is now H one knows this window is full.

7:29

So as if this window is full, there is no point of sending data over here.

7:34

So here see what is happening.

7:36

You you see here advertisement of window size that that host are doing during transmission.

7:45

And by this advertisement of window size what happens is transmitter will get to know like okay, this

7:53

much size of buffer that is available at receiver side so that that transmitter will not forward data,

8:00

which is more than that available buffer size.

8:03

Right now you see there can be two scenarios which is possible over here.

Persistent Timer to Avoid Deadlock

8:10

As if this window size that is full, then here, this H1.

8:16

Now it will stop sending the data over here.

8:19

Right.

8:20

But obviously after some time this data that will be given to application layer see transport layer will

8:27

give data to application layer at the receiver side.

8:30

Right.

8:31

So after some time this buffer data that will be given to application layer.

8:36

So once the data that is given to application layer this window will get empty by some extent like you

8:42

see here.

8:44

this H2 is having window which is available up to 1300 byte right now.

8:52

So what h2 will do?

8:54

H2 will do.

8:55

Advertisement of window size over here.

8:58

H2 will say like okay, my window is available by 1300 byte.

9:05

But what if this frame.

9:07

What if this segment is not received at H1?

9:11

As if this segment is not is not received at H1, then H1 will never come to know like okay, now this

9:18

window is available over here.

9:20

So here there will be deadlock.

9:23

Here there will be deadlock.

9:24

Why there will be deadlock.

9:25

The reason is H1 is stop sending.

9:28

H1 is not sending data.

9:30

H1 is not sending data because of what reason?

9:33

Because of H1 knows this window is full right now at H2 side.

9:39

But after some time H2 is saying like okay, my window is available up to 1300 bytes, but that frame

9:46

is not received over here.

9:48

And because of this frame is not received over here H1 will never get to know.

9:54

Like okay, I can forward data in this direction to H2.

9:59

So there will be deadlock if this segment is not getting received over here.

10:04

Obviously if this segment is getting received then from here onwards, from here onwards H1 will start

10:12

to forward the data.

10:13

So here there is very interesting concept which is persistent timer by which this deadlock that we can

10:21

avoid.

10:22

You see how see after some time means after persistent timer.

10:28

After some time H1 will forward one frame in that there will be no data, in that there will be no data.

10:36

It will be a probe frame that you can see.

10:40

So in that probe frame it will say this H2 are you free?

10:46

Are you having available window size.

10:49

Then based on this frame, H two will give reply.

10:53

Okay, right now I am having complete window which is free, which is having size of 2000 byte.

10:59

So once this feedback received by H1, H1 will forward the data from here onwards.

11:07

But you see here persistent timer that we are setting.

11:12

Why?

11:12

The reason is we want to avoid deadlock.

11:16

See if you if you communicate completely based on this acknowledgement.

11:20

In that case what will what is possible in that case.

11:24

As if this acknowledgement is not received by transmitter, in that case, there can be deadlock.

11:32

Deadlock means this H one will never get to know, like okay, I should be transmitting data from here

11:37

onwards, right?

11:39

So that is persistent timer concept.

11:42

After that, this H one will forward probe frame in which it will ask H two are you free now?

11:50

I have waited for some time period, and after that you should be having some space in your window.

11:56

So H2 will reply.

11:58

Okay, now I am having space of 2000 byte.

12:00

You can forward the data.

12:02

So that is how three essential concepts that you need to understand in flow control.

12:07

First is first is window scaling.

12:11

So window scaling that we do initially by giving details of window size at H1 and H2 size.

12:18

And both of these computer will get agreed upon lower of those two window size right.

12:25

Minimum of those two window size.

12:28

And then they will be initiating communication.

12:31

During communication there will be acknowledgment frame by receiver like you see acknowledgment frame

12:38

by receiver in that it will be doing advertisement of window size.

12:43

Advertisement of window size means it will say like this much window that is available with me.

12:48

If there is no space that is available.

12:51

In that case, transmitter will have to stop sending off that data.

12:56

Right.

12:57

And once that window is available, then the receiver will give feedback in which it will say like,

13:06

okay, this much size is available.

13:07

You can forward the data, but there is always possibility of deadlock.

13:11

As if this type of frame that is getting lost somewhere in the computer network.

13:15

So here there is a concept of persistent timer.

13:18

After persistent timer, there will be pro frame that is getting forwarded by the transmitter.

13:24

And based on that again we can resume the communication.

13:28

Right.

13:28

So we are not only depending on that window or advertisement, we are also having persistent timer concept

13:34

which will avoid deadlock situation in communication.

13:39

So this is how complete flow control mechanism that is happening in TCP.

13:43

Still if you have any query please note it down in comment

section

Let's break down and **explain the diagram** you provided, which shows how **TCP flow control** works using *window scaling*, *advertised window size*, and the *persistent timer* mechanism to avoid deadlock.

13:47

I'll be happy to help you.

13:47

Thank you so much for watching this video.

explain

Diagram Overview

- H1 and H2 are two computers communicating using TCP.
- W1: Window size for H1 (sender)
- W2: Window size for H2 (receiver)
- The arrows show communication between the hosts, including window size negotiations, data transfer, and flow control events.

Step-by-Step Explanation

1. Window Scaling and Connection Establishment

- **Request, $W1 = 3000B$:**

H1 tells H2 that it can handle a window size (buffer) of **3000 bytes**.

- **Reply, $W2 = 2000B$:**

H2 replies that its window size is **2000 bytes**.

- **Agreement:**

The *effective window size* is the **smaller** of the two, so both agree to use **2000 bytes** ($W = \min(3000, 2000) = 2000$).

2. Data Transmission and Window Advertisement

- **Data Transfer:**

H1 starts sending data to H2, which fills up H2's buffer.

H2 updates its advertised window size ($W2$) after each data reception:

- After some data: **$W2 = 1000B$** (only 1000 bytes buffer free)
- After more data: **$W2 = 0B$** (buffer is full)

- **Window Advertisement:**

H2 notifies H1 of the remaining buffer with each ACK (acknowledgement).

3. Flow Control in Action

- **Stop Sending:**

Once H2's buffer is full (**$W2 = 0B$**), H1 must *stop sending* more data to avoid overflow.

- **Persistent Timer:**

H1 starts a *persistent timer*—a countdown to periodically check if it can resume sending data.

4. Receiver Application Consumes Data

- **Buffer Frees Up:**

The application on H2 reads data from the buffer, making space:

- H2 can now accept **1300 bytes** (**$W2 = 1300B$**).

- **Window Advertisement Lost:**

H2 sends an updated window advertisement (**$W2 = 1300B$**) to H1, but this message is **lost** in the network.

5. Deadlock Scenario

- **No More Sending:**
H1 keeps waiting because, as far as it knows, H2's buffer is still full (**W2 = 0B**).
- **Deadlock:**
Both sides are stuck—H1 waiting to send, H2 waiting to receive—because H1 never gets the updated window advertisement.

6. Persistent Timer Saves the Day

- **Probe (IB, Prob):**
When the *persistent timer* expires, H1 sends a **probe message** (a segment with zero data) to ask H2 if it has free buffer space.
- **H2 Replies (W2 = 2000B):**
H2 responds with its current window size (now **2000 bytes** free).
- **Data Transfer Resumes:**
H1 resumes sending data based on the new window size.

Key Concepts Illustrated

1. Window Scaling

- Both hosts tell each other their buffer sizes, and the minimum is used to avoid buffer overflows.

2. Advertised Window Size

- The receiver (H2) constantly advertises its available buffer space so the sender (H1) knows how much data it can safely send.

3. Flow Control

- Sender (H1) must stop sending if receiver's buffer is full, and only resumes when notified of free space.

4. Deadlock and Persistent Timer

- If the "buffer available" message is lost, sender might wait forever (*deadlock*).

- **Persistent timer** ensures that sender will periodically check if receiver is ready again, even if some messages are lost.
-

Simple Analogy

Imagine two people passing books (data):

- The receiver has a limited shelf (buffer).
 - When the shelf is full, the sender stops sending books.
 - The receiver says, "I have space now!" but if that message is lost, the sender waits forever.
 - Every few minutes, the sender asks, "Is there space now?" This is the **persistent timer** mechanism.
-

Summary Table

Event	Action
H1, H2 exchange window sizes	Use minimum window size
H1 sends data	H2 buffer fills up, advertises available space
H2 buffer full	H2 advertises 0B; H1 stops sending
H2 buffer frees up	H2 advertises new space (may get lost!)
H1 receives update	Data transfer resumes
H1 doesn't receive update	Deadlock unless persistent timer triggers a probe
Persistent timer triggers	H1 sends probe; H2 responds with available buffer size
H1 resumes sending	After receiving updated buffer info

Conclusion

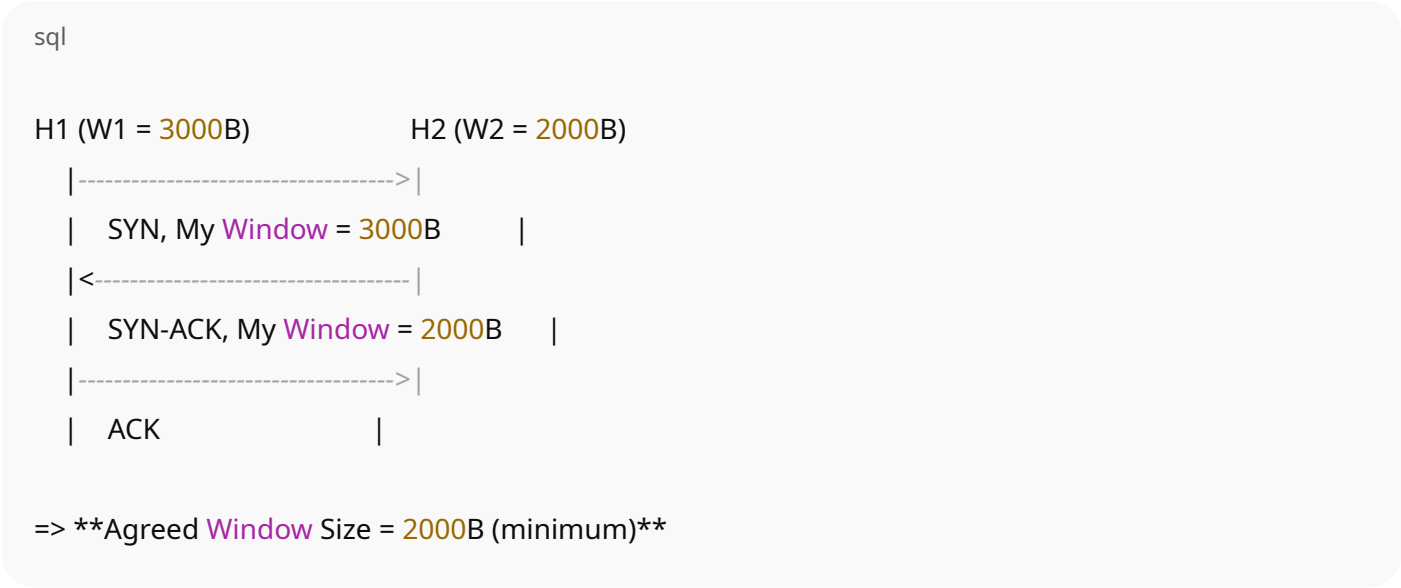
- **TCP flow control** prevents buffer overflow using window sizes.
- **Advertised window** keeps the sender informed about receiver's available space.
- **Persistent timer** ensures recovery from lost window advertisements (avoiding deadlock).

Let me know if you need a diagram, a math example, or further details on any specific part!

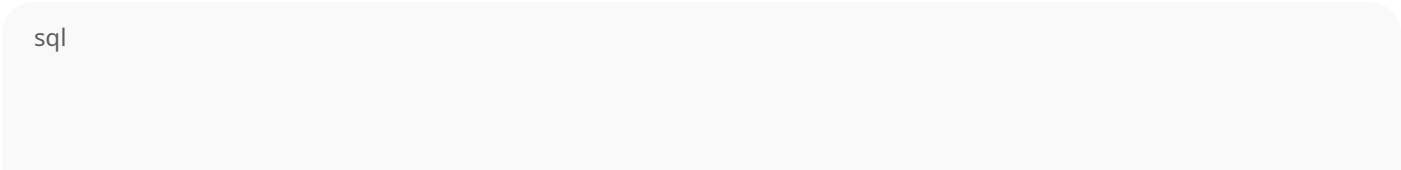
show me detailed diagram of what is happening here

Absolutely! Here's a **step-by-step detailed diagram** of what's happening in the flow control scenario you provided. This will visually clarify each phase, window size changes, ACKs, and the deadlock situation.

Step 1: Connection Establishment (Window Negotiation)



Step 2: Data Transfer Begins




```
H1 (W1 = 2000B)          H2 (W2 = 2000B)
|----DATA (2000B)----->|
|                          | [Buffer: 2000B filled]
|<---ACK, Window = 0B-----|
(H2 says: Buffer full, STOP sending!)
```

Step 3: Buffer Becomes Partially Free

```
sql

H1 (W1 = 2000B)          H2 (W2 = 0B)
| (Waiting...)           | [Application reads 1000B]
|                          | [Buffer: 1000B used, 1000B free]
|<---ACK, Window = 1000B-----|
(H2 advertises 1000B free space)
```

Step 4: H1 Sends More Data, Buffer Fills Again

```
sql

H1 (W1 = 2000B)          H2 (W2 = 1000B)
|----DATA (1000B)----->|
|                          | [Buffer: 2000B filled again]
|<---ACK, Window = 0B-----|
(Buffer full, H1 must STOP)
```

Step 5: Application Frees Buffer at H2, But Window Update is Lost

```
sql

H1 (W1 = 2000B)          H2 (W2 = 0B)
| (Waiting...)           | [Application reads 1300B]
```

```
| [Buffer: 700B used, 1300B free]
|<---ACK, Window = 1300B---X
(This update is **lost in network**)
```

Step 6: DEADLOCK!

- H1 **never receives** the window update (still thinks W2 = 0B).
- H2 is ready to receive, but H1 is waiting forever.

less

```
H1 (W1 = 2000B)      H2 (W2 = 1300B)
| (Still waiting for space) | (Has free space but H1 doesn't know!)
|-----|
DEADLOCK!
```

Step 7: Persistent Timer & Probe

Persistent timer expires at H1:

lua

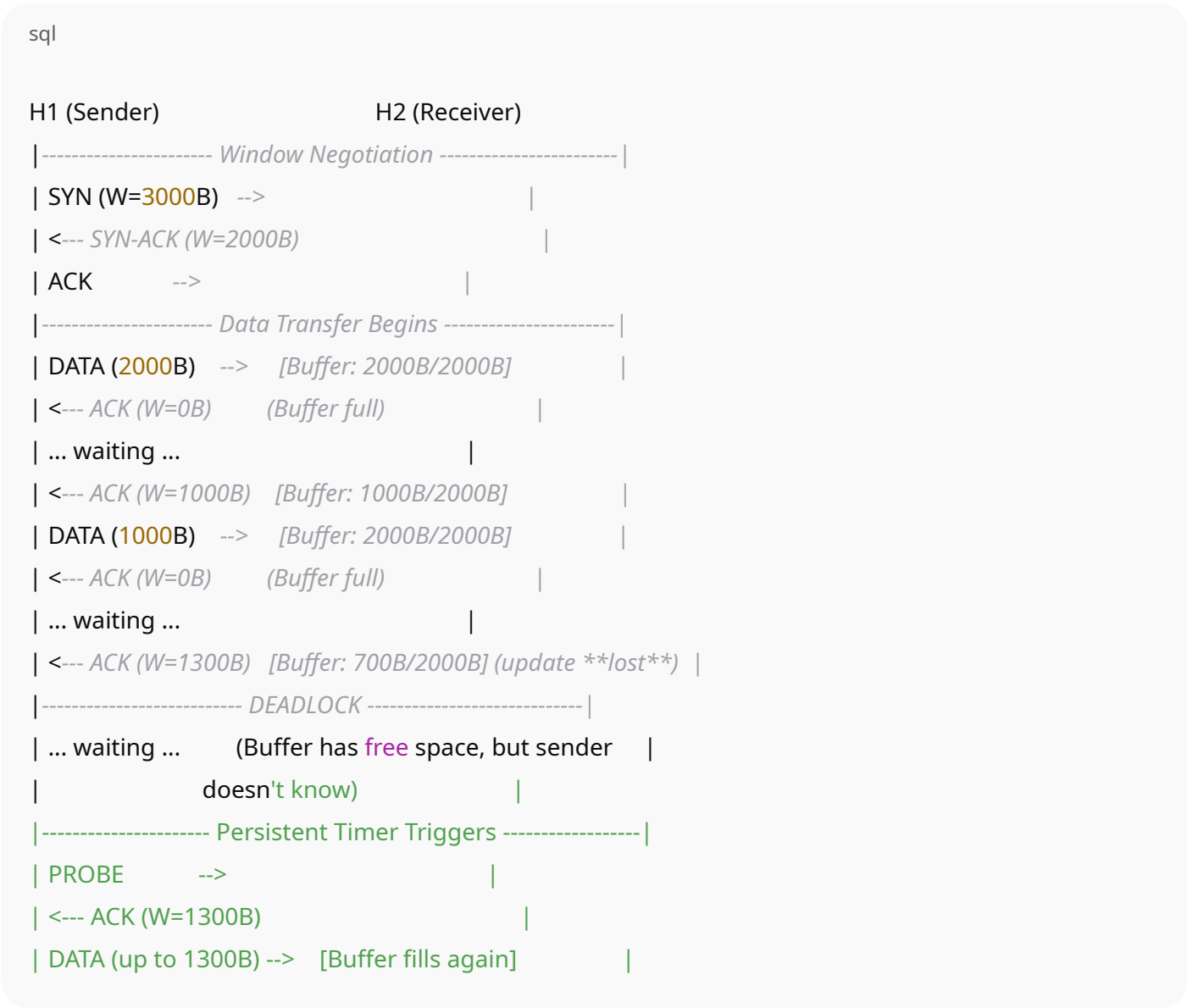
```
H1 (W1 = 2000B)      H2 (W2 = 1300B)
|---PROBE SEGMENT----->|
|                          |
|<---ACK, Window = 1300B-----|
|                          |
|---DATA (up to 1300B)----->|
|                          | [Buffer fills again]
```

Step 8: Communication Continues

Once the window update is received, H1 resumes sending data, and the process repeats as needed.

Full Visual Flow

Below is a **timeline diagram** summarizing all steps:



Visual Summary

- **Black arrows:** Data transfer
- **Blue arrows:** Window size updates (ACKs)

- **Red X:** Lost window update (causes deadlock)
 - **Green arrows:** Probe segment from persistent timer
-

If you want an actual illustrated (image) diagram instead of ASCII art, let me know—I'll draw and upload it for you!

Would you like an image? Or is this step-by-step text/ASCII diagram enough?